

1. Two Sum

Easy · Array · Hash Table

Approach — One-pass Hash Map

The brute-force way to solve this problem is to check every pair of numbers, which takes $O(n^2)$ time. A much faster approach is to use a **hash map** to remember the numbers already seen, along with their indices, while scanning the array just once.

For each number `nums[i]`, the value needed to reach the target is `target - nums[i]`.

Before inserting the current number into the hash map, we check whether this complement already exists in the map. If it does, we have found the pair immediately — no need to look ahead or revisit elements.

Key Insight: Because the complement is checked *before* the current number is added to the map, the two indices returned are guaranteed to be distinct, satisfying the problem's constraint that an element cannot be used twice with itself.

Step-by-step Breakdown

Step 1 — Initialize the hash map

Create an empty `unordered_map` called `hashMap` that will store each visited number as a key and its index as the value.

Step 2 — Compute the complement

For the current index `i`, calculate the value needed to reach the target: `target - nums[i]`. This is the number we are hoping to find already in the map.

Step 3 — Check for a match

Search the hash map for the complement using `hashMap.find(target - nums[i])`. If found (`it != hashMap.end()`), the indices of the complement (`it->second`) and the current element (`i`) form the answer — return them immediately.

Step 4 — Record the current number and continue

If no match was found, store the current number and its index in the map with `hashMap[nums[i]] = i`, then move to the next element. If the loop finishes without finding a pair, return an empty vector.

Solution Code

```
1  class Solution {
2  public:
3      vector<int> twoSum(vector<int>& nums, int target) {
4          unordered_map<int, int> hashMap;
5          for(int i = 0; i < nums.size(); i++)
6          {
7              auto it = hashMap.find(target-nums[i]);
8              if(it != hashMap.end())
9              {
10                 return {it->second, i};
11             }
12             hashMap[nums[i]] = i;
13         }
14         return {};
15     }
16 };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
$O(n)$	$O(n)$
The array is scanned once. Each hash map lookup and insertion takes $O(1)$ amortized time, giving an overall linear pass.	In the worst case, all n elements are inserted into the hash map before a match is found, requiring $O(n)$ extra space.